OS ORANGE PROBLEM
(BUILDING PES-VCS)

Name: Akshat Kumar

SRN: PES2UG24CS046

SECTION: 4 A

Screenshots:

PHASE 1:

1A)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bb
e9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea0922
7ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

1B)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda47
1e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

PHASE 2:

2A)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
In file included from tree.c:12:
index.c: In function ‘index_add’:
index.c:238:9: warning: implicit declaration of function ‘object_writ’
 [-Wimplicit-function-declaration]
  238 |     if (object_write(OBJ_BLOB, buffer, size, &id) != 0) {
      |         ^~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

2B)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda47
1e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68
a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

PHASE 3:

3A)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ echo "hello" > file1.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ echo "world" > file2.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes add file1.txt file2.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  test_sequence.sh
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  PES2UG24CS046-pes-vcs
  untracked:  README.md
  untracked:  index.c
  untracked:  tree.h
  untracked:  tree.c
  untracked:  index.h
  untracked:  test_tree
  untracked:  object.c
  untracked:  commit.c
```

3B)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb
4 1776399070 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7
f 1776399085 6 file2.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

PHASE 4:

4A)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ echo "Hello" > hello.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes add hello.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes commit -m "Initial commit
"
Committed: a82bd8f9677f... Initial commit
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ echo "World" > hello.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes add hello.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 99f414e78167... Add farewell
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ echo "Goodbye" > bye.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes add bye.txt
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 921c53ea216f... Add farewell
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ ./pes log
commit 921c53ea216f32303a4878ce3bd2c3ecd80c5bd2953ffe5c00aa928b9ee2866
8
Author: PES User <pes@localhost>
Date:   1776399372

    Add farewelldd fQ

commit 99f414e7816739c362bcbf809ce0b9b9b03abf01bb36131f774952af2a7d82b
c
Author: PES User <pes@localhost>
Date:   1776399334

    Add farewelldd fQ

commit a82bd8f9677fda24000b2449a41cf2e71987ebb19db9a51dc8786eb9c329af7
0
Author: PES User <pes@localhost>
Date:   1776399267

    Initial committial commiQ
```

4B)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/15/48474abcf142b8e59a06022f0e26f8c51a97fb530ff5efad9dc797
e251ad38
.pes/objects/1a/488fa7ecf11e22c69ac42591b5019850185ba5ff9e9fba6fd049cf
7204af0d
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda47
1e566fcf
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de57
6db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8
007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974b
e6191900
.pes/objects/92/1c53ea216f32303a4878ce3bd2c3ecd80c5bd2953ffe5c00aa928b
9ee28668
.pes/objects/99/f414e7816739c362bcbf809ce0b9b9b03abf01bb36131f774952af
2a7d82bc
.pes/objects/a8/2bd8f9677fda24000b2449a41cf2e71987ebb19db9a51dc8786eb9
c329af70
.pes/objects/b3/d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc48
78ebec12
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
b9aafc12
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f6
06199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13
a68652bc
.pes/refs/heads/main
```

4C)

```
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ cat .pes/refs/heads/main
921c53ea216f32303a4878ce3bd2c3ecd80c5bd2953ffe5c00aa928b9ee28668
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

FINAL:

```
refs/heads/main
akshat@Akshat:~/PES2UG24CS046-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function ‘tree_from_index’:
tree.c:201:13: warning: implicit declaration of function ‘object_writ’ [-Wimplicit-function-declaration]
  201 |         if (object_write(OBJ_TREE, subdata, sublen, &sub_id) != 0) {
      |             ^~~~~~~~~~~~~
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: cb73af716ab3... Initial commit

Log after first commit:
commit cb73af716ab35548b4dd58f11bdfdfe6011d4ad72afcb0e1b38337ee8ad1b00f
Author: PES User <pes@localhost>
Date:   1776403387

    Initial committial commiQ


--- Second Commit ---
Committed: 42bac1f9f93e... Update file.txt

--- Third Commit ---
Committed: f6213e61affc... Add farewell

--- Full History ---
commit f6213e61affc834c70fc59388738564b08d57a93cbca4568281d3c56aaf0a675
Author: PES User <pes@localhost>
Date:   1776403387
```

```
    Initial committial commiQ


--- Second Commit ---
Committed: 42bac1f9f93e... Update file.txt

--- Third Commit ---
Committed: f6213e61affc... Add farewell

--- Full History ---
commit f6213e61affc834c70fc59388738564b08d57a93cbca4568281d3c56aaf0a675
Author: PES User <pes@localhost>
Date:   1776403387

    Add farewelldd fQ

commit 42bac1f9f93e74b9f3e89da31435ff9b1e6198ee1abcc6832ab0a289438ef7aa
Author: PES User <pes@localhost>
Date:   1776403387

    Update file.txtfQ

commit cb73af716ab35548b4dd58f11bdfdfe6011d4ad72afcb0e1b38337ee8ad1b00f
Author: PES User <pes@localhost>
Date:   1776403387

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f6213e61affc834c70fc59388738564b08d57a93cbca4568281d3c56aaf0a675

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/42/bac1f9f93e74b9f3e89da31435ff9b1e6198ee1abcc6832ab0a289438ef7aa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/cb/73af716ab35548b4dd58f11bdfdfe6011d4ad72afcb0e1b38337ee8ad1b00f
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f6/213e61affc834c70fc59388738564b08d57a93cbca4568281d3c56aaf0a675

=== All integration tests completed ===
akshat@Akshat:~/PES2UG24CS046-pes-vcs$
```

PHASE 5 & 6 (ANALYSIS QUESTIONS):

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement `pes checkout <branch>`, the file `.pes/refs/heads/<branch>` is read to get the commit hash, and `.pes/HEAD` is updated to point to that branch. The working directory must then be updated to match the tree of that commit by recreating all files and directories from the object store. The index must also be updated accordingly. This operation is complex because it involves safely overwriting files, handling directory structures, and ensuring uncommitted changes are not lost.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

For each file in the index, compute the current hash from the working directory and compare it with the stored hash in the index. If they differ, the file has uncommitted changes. Then compare the index version with the version in the target branch’s tree from the object store. If both differ, a conflict exists. In such cases, checkout must be refused to prevent overwriting user changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, new commits are created normally but are not referenced by any branch. HEAD moves forward, but no branch points to these commits, making them potentially unreachable later. A user can recover them by creating a new branch pointing to the current HEAD or by manually saving the commit hash in a branch reference.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Start from all branch heads and add their commit hashes to a set of reachable objects. Traverse each commit to collect its parent and tree, and traverse trees to collect all referenced blobs and subtrees. Continue until all reachable objects are found. Then delete any objects in `.pes/objects` not present in this set. A hash set should be used for efficient lookup. For 100,000 commits, the total objects visited could be around 300,000 to 500,000 including commits, trees, and blobs.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

It is dangerous because a commit operation may create new objects that are not yet referenced by any branch. If garbage collection runs at the same time, it may consider these objects unreachable and delete them. This creates a race condition where a commit refers to objects that no longer exist. Git avoids this by using locks and by only deleting objects that are safely unreachable for a certain period, ensuring active operations are not affected.